

Ikkat System Design Decisions

- We have chosen a primary backup model
 - This means that only the leader has the ability to handle requests such as `create`, `write`, `open`, `delete`, `close`
 - Followers replicate the state and act as backup
 - Due to the robust backup system, even though there is just one leader, if the leader fails, the system will be able to easily recuperate and continue its normal functioning
- Replication is implemented using log files
 - The leader writes to a local log, which is later sent to followers. After a majority acknowledgement is reached, the log is committed
 - This is a simplified implementation of the Raft system
 - Write consistency is ensured by the majority voting
- Fault tolerance is implemented in many ways, such as
 - Incomplete writes being deleted after a crash
 - Furthermore, entry and lock conditions are reset, preventing deadlock
 - Heavy usage of `s.mu` and `fm.mu`, which lock the system at important points, such as getting access to files
 - Heartbeats are used to ensure liveness
 - The servers maintain a map of the active nodes
 - If a heartbeat is missing, the node is marked as dead
 - Since the size of the system is small, such a solution works quite effectively
 - Concurrency is controlled via locks and leases. Files' access can only be given if the access read/write and can read/write tokens are passed to a particular node
 - Using locks ensures correctness since only one node can have the access at a time
 - Having the lease system allows the server to clean up nodes that crashed while accessing a particular file
 - Leases are released once the task is finished. The function `cleanupLeases` makes sure that if a node fails while holding a lease, it will be returned back to the system
- New server initializations are put in backup position, which is so that there aren't multiple leaders
- Idempotency is implemented using checking the cache of requests
- The prime set is rebuilt in case of crashes, which allows the logic to be cleaner as well as storing the data
 - However, the re-computation is quite intense.

- This increase in computation feels justified from our end as output files would be much smaller than the input files, and this approach felt like the simplest to implement
- Guaranteeing atomicity in a distributed system is quite hard. Our approach was to have the node write to a temporary file, which is then copied to the actual file once the function is complete. This allows us to have both atomicity and crash safety
- Every update is logged using the `appendToDisk` function, which is then replayed on restart using the `recoverFromLog`
 - This allows the system to recover in case of crashes, as it can reconstruct the state by reading the system
 - This is how we implemented Write Ahead Logging
- File reading is done via 4KB chunks, which is both memory efficient and scalable for the prime finding application
- The client maintains cache using dirty writes and cache eviction, which improves performance
 - This means that the client keeps a local copy, which allows the client to access
 - Dirty writes allow for better performance, as once the chunk is no longer in use, all the edits can quickly be combined and then written into the file
 - Cache eviction ensures that the client side memory is controlled and that repeat requests don't end up slowing down the system
- Periodically, using the `cleanupRequests` and `cleanupLeases` functions, the system cleans up failed nodes, memory leaks and just generally allowing the system to function better
- To implement snapshots, we are taking advantage of the fact that our system locks itself for implementing function such as `cleanupLeases`. The leader node will send a message to all clients, locking the system and asking to record the state and send it to the leader, which will then be recorded and presented as a snapshot.